

Why I Switched to Polars From Pandas

Benchmarking Polars vs Pandas DataFrames

AUTHOR
Russ Zaliznyak <rzaliznyak@gmail.com>

PUBLISHED
October 20, 2025



TL/DR;

If you use **Pandas** dataframes to store and analyze large datasets, consider making a change to **Polars**.

I use *dataframes* because they allow me to think relationally — just like I do when I write SQL. I use them extensively in virtually all of my work; app development, exploratory data analysis, and summarizing large datasets.

My colleague Joseph Powers, PhD, introduced me to **Polars** — claiming it offers big runtime savings over **Pandas**. I scoffed, but soon enough, processing pandas DFs became such a bottleneck that I had no choice but to switch.

Check out below the benchmarks I measured for typical dataframe operations like read, write, construct, and group by. These will convince you that a change is necessary.

System & Environment

Methodology

I benchmarked **Polars** and **Pandas** across common DataFrame operations using both real and simulated data.

- For **read** and **write** tests, I used a **10 MB CSV dataset** from the [Frictionless Data repository](#).

- For **constructor**, **pivot**, **join**, and **filter** benchmarks, I generated a **simulated dataset** to ensure consistent structure and reproducibility.
- Each operation was executed **** 10 times**** per library, and the **mean execution time** was recorded.

Read | Write

Download **10 mb file**: <https://raw.githubusercontent.com/frictionlessdata/datasets/main/files/csv/10mb.csv>

Metric	Mean Speed Increase	Standard Deviation
Read	16.24	2.47
Write	24.26	7.21

AI Generated Summary

Polars is faster at reading and writing CSVs than pandas because it's built in Rust with parallelized, memory-efficient algorithms that fully utilize multiple CPU cores.

Construct | Pivot

► Simulate Dataset

Metric	Mean Speed Increase	Standard Deviation
Constructor	3.47	0.07
Group By	9.45	2.24

AI Generated Summary

Constructor: Polars DataFrame constructor is faster than pandas because it uses Rust's zero-copy memory model and parallel data parsing, avoiding Python's object overhead.

Pivot Tables: Polars outperforms pandas in pivot table operations by executing grouped aggregations in parallel and leveraging an optimized query engine built on Apache Arrow for efficient columnar computation.

Joins | Filter

Metric	Mean Speed Increase	Standard Deviation
Inner Join	8.78	6.73
Union All	0.32	0.24
Filter	0.53	0.61

AI Generated Summary

Joins (Inner + Union All): Polars performs joins faster than pandas by using parallelized hash join algorithms implemented in Rust and operating directly on Arrow memory, minimizing copying and serialization overhead.

Filter Clauses: Polars evaluates filter expressions lazily and in parallel at the columnar level, enabling vectorized, cache-efficient filtering that significantly outpaces pandas' row-wise execution in Python.

Conclusion

Across the board **Polars** is faster at all of the typical dataframe operations we tested. The syntax differences are minimal and AI can quickly convert your **Pandas** code. Make this transition and speed up your work!

Code References

Operation	Pandas Example	Polars Example	Times Faster
Read CSV	<code>df = pd.read_csv("data.csv")</code>	<code>df = pl.read_csv("data.csv")</code>	16.24
Write CSV	<code>df.to_csv("out.csv")</code>	<code>df.write_csv("out.csv")</code>	24.26
Constructor	<code>df = pd.DataFrame(data)</code>	<code>df = pl.DataFrame(data)</code>	3.47
Group By	<code>df.groupby("col")["value"].mean()</code>	<code>df.groupby("col").agg(pl.col("value").mean())</code>	9.45
Inner Join	<code>df1.merge(df2, on="id", how="inner")</code>	<code>df1.join(df2, on="id", how="inner")</code>	8.78
Union All	<code>pd.concat([df1, df2], ignore_index=True)</code>	<code>df1.vstack(df2)</code>	0.32
Filter	<code>df[df["value"] > 10]</code>	<code>df.filter(pl.col("value") > 10)</code>	0.53

*** Results will vary depending on machine and size/structure of your datasets.

AI Generated Summary: Why Union All Has High Variance

AI Generated Summary: Why Constructor is Reliably Faster

The constructor benchmark is the most reliable because it's a **pure in-memory, CPU-bound operation** with no I/O, threading, or garbage collection noise. Each run follows the same predictable memory layout, making it a clean measure of Polars' and Pandas' internal efficiency without interference from system variability.